

TF - Final

CMP223 - Computer System Performance Analysis

Prof. Dr. Lucas Mello Schnorr

João Luiz Grave Gross

01 de dezembro de 2025

Roteiro

- Plataforma: Especificações e Acesso
- Características das implementações
- Cenários de Testes
- Variáveis de Ambiente
- Projeto Experimental
- Resultados
- Conclusões
- Reprodutibilidade

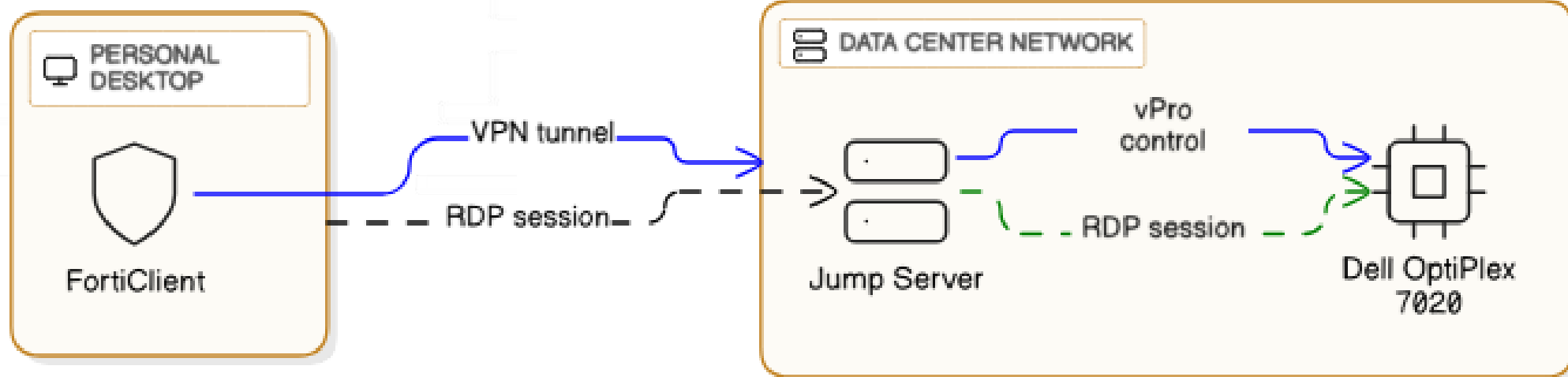
Plataforma: Especificações e Acesso



OptiPlex Micro

Modelo: 7020

- 14ª geração Intel® Core™ i7-14700T vPro® (20 core, 28 threads, cache de 33MB, 1.3 GHz a 5.0 GHz)
- 16GB DDR5 (1x16GB) 5600MT/s
- SSD de 512GB PCIe NVMe M.2
- OptiPlex Micro com CPU de 35W



Características das implementações

- **Python 3.12.12 – Multiprocessing**

- 8 processos filhos criados com `fork()` → Set core CPU Affinity (**OK**)
- Herdam memória do pai: Copy-On-Write (CoW)
 - **SE** houver **escrita** há transferência de dados → Só há **leitura** (**OK**)

- **Python 3.12.12 – Dask**

- Processo pai: Limita a Poll de 8 núcleos
- 8 tarefas criadas → Dask Scheduler define núcleos dos Workers (dos 8 do pai)
- Comando **da.matmul** não permite Set core CPU Affinity

- **Python 3.14.0t (free-threading) - Threading**

- Sem GIL 🎉 🎉 → Sem trava de processamento sequencial
- Memória compartilhada entre todas as 8 threads → Zero Copy
- Set core CPU Affinity (**OK**)

Cenários de Testes

- **8P_HT_OFF**
 - 8 P-Cores
 - Hyper-Threading OFF (1 thread por core)
- **4P_HT_ON**
 - 4 P-Cores
 - Hyper-Threading ON (2 threads por core)
- **4P_4E_HT_OFF**
 - 4 P-Cores → Hyper-Threading OFF (1 thread por core)
 - 4 E-Cores
- **8E**
 - 8 E-Cores → 1 thread por core

Variáveis de Ambiente

- **Para Python 3.12.12**

- OMP_NUM_THREADS=1 MKL_NUM_THREADS=1
VECLIB_MAXIMUM_THREADS=1 OPENBLAS_NUM_THREADS=1
NUMEXPR_NUM_THREADS=1 **jupyter notebook**

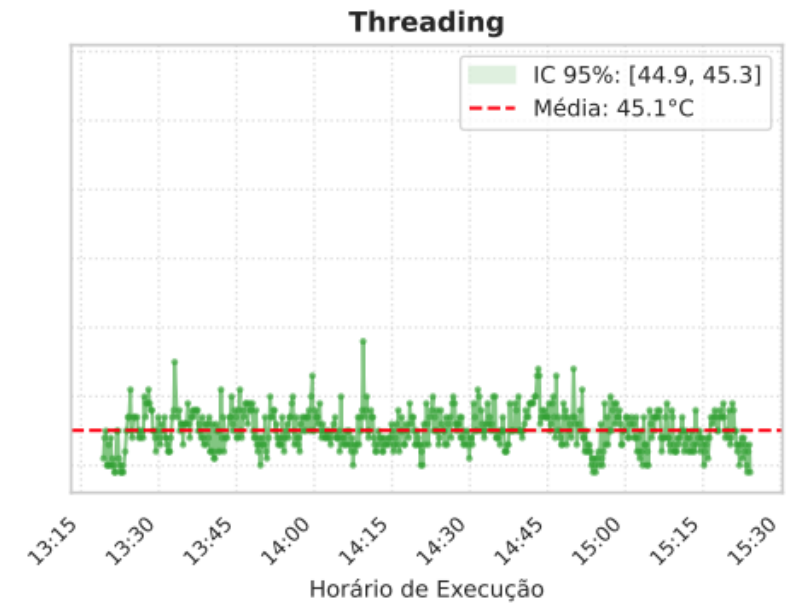
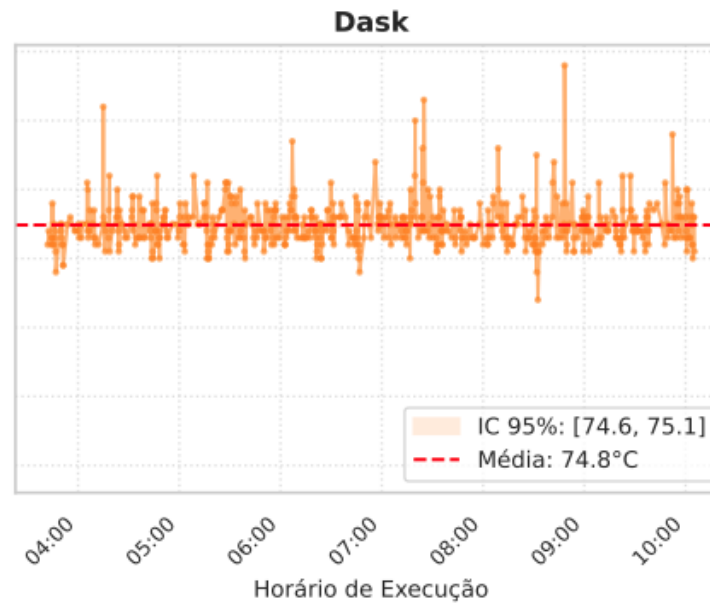
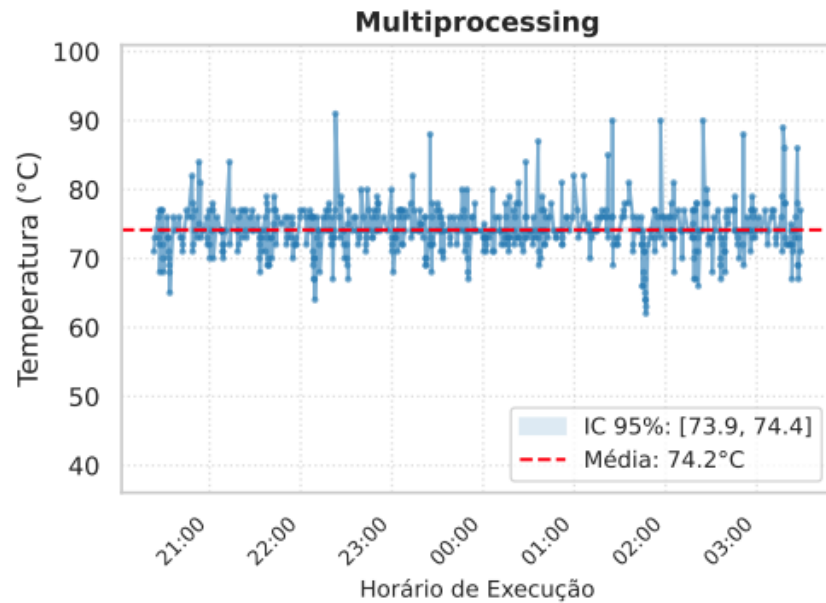
- **Para Python 3.14.0t**

- OMP_NUM_THREADS=1 MKL_NUM_THREADS=1
VECLIB_MAXIMUM_THREADS=1 OPENBLAS_NUM_THREADS=1
NUMEXPR_NUM_THREADS=1 **PYTHON_GIL=0 jupyter notebook**

Projeto Experimental

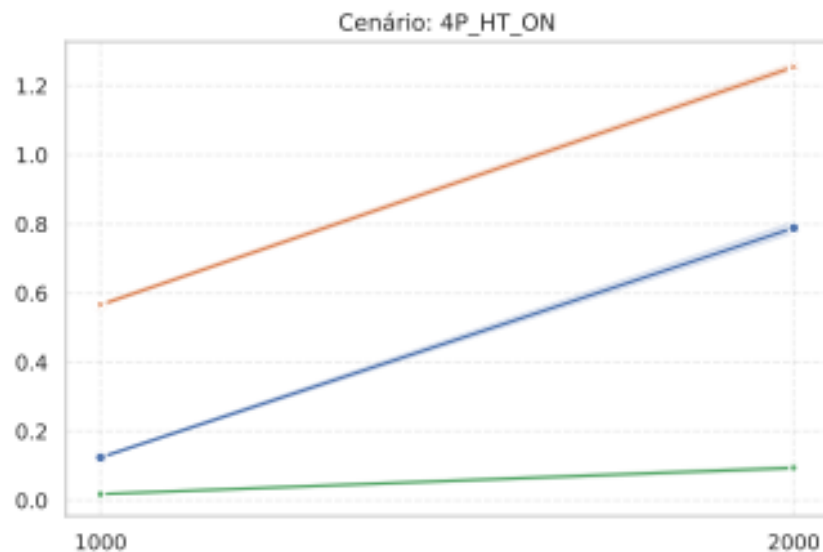
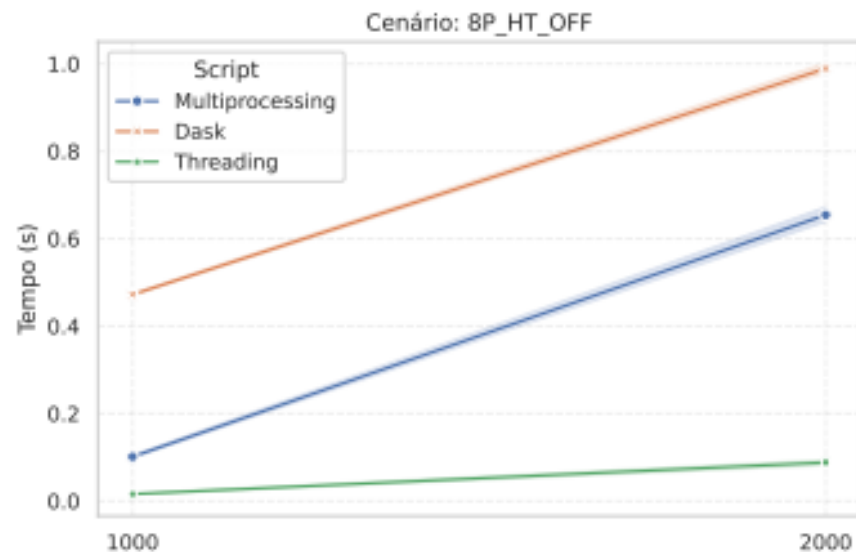
- **Tamanho de Matriz:** 1000, 2000, 4000, 6000, 8000
 - **Cenários:** 8P.HT.OFF, 4P.HT.ON, 4P.4E.HT.OFF, 8E
 - **Réplicas:** 30 repetições
- Configurações: [Matrix_Size: 5] X [Cenários: 4] X 30 = 600
-
- Cada script executa o projeto experimental uma vez
- **Total:** [warm-up + 600] x 3 scripts = 1800 runs

Resultados: Evolução da Temperatura



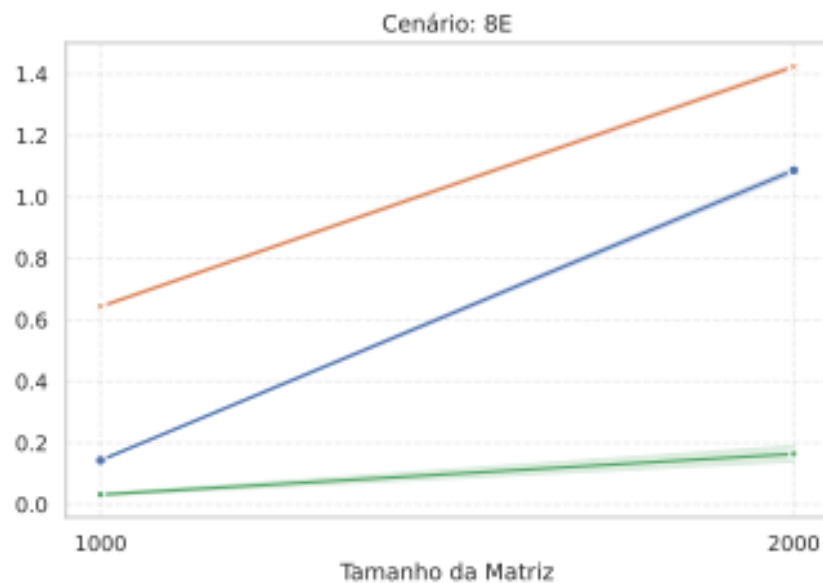
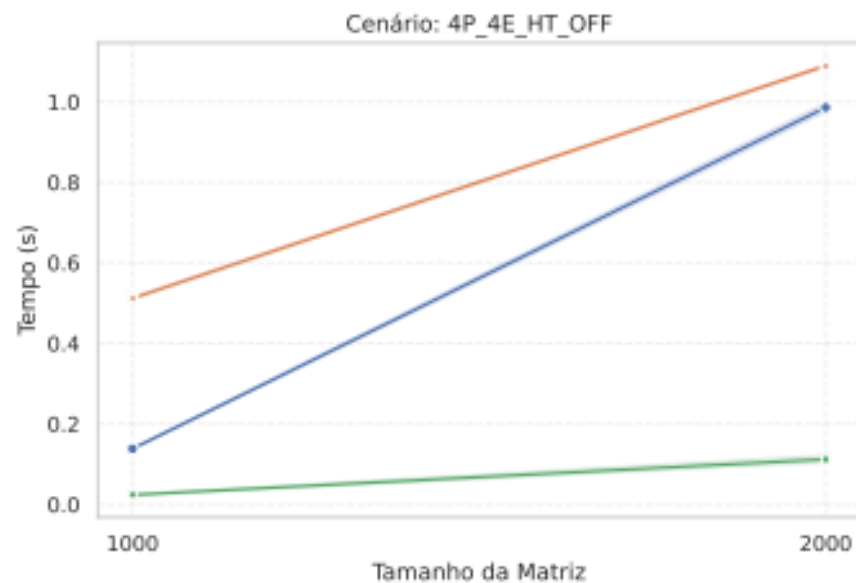
- Multiprocessing e Dask: 74.2°C e 74.8°C
 - Manter múltiplos contextos de execução
- Threading: 45.1°C
 - Menos demanda à CPU

Resultados: Por Cenários – Matriz 1000 e 2000



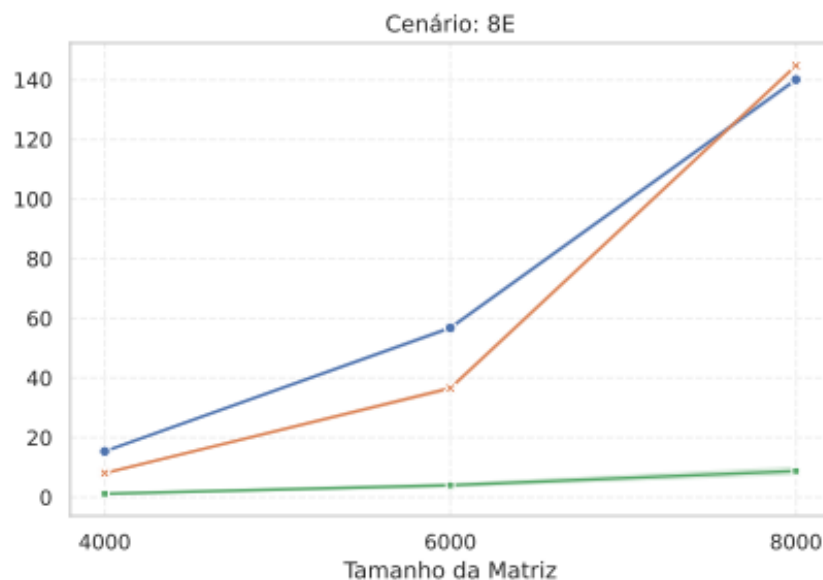
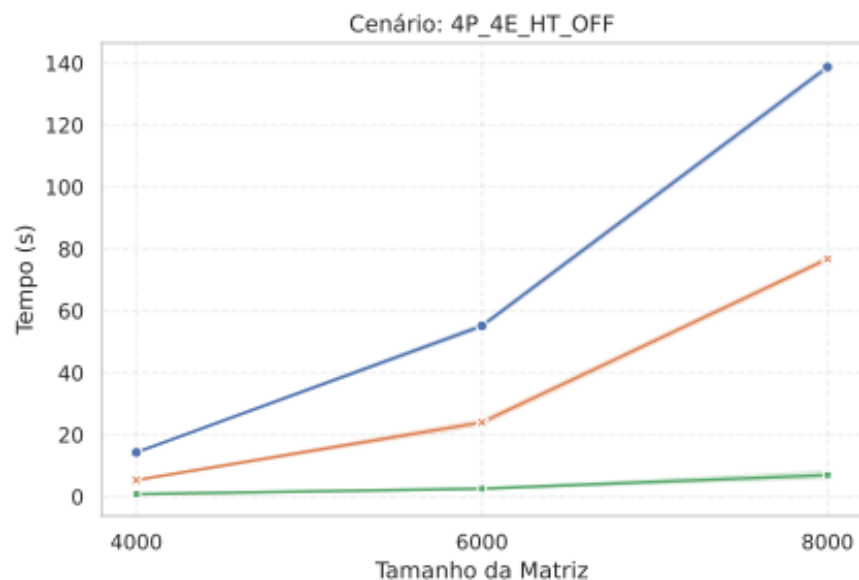
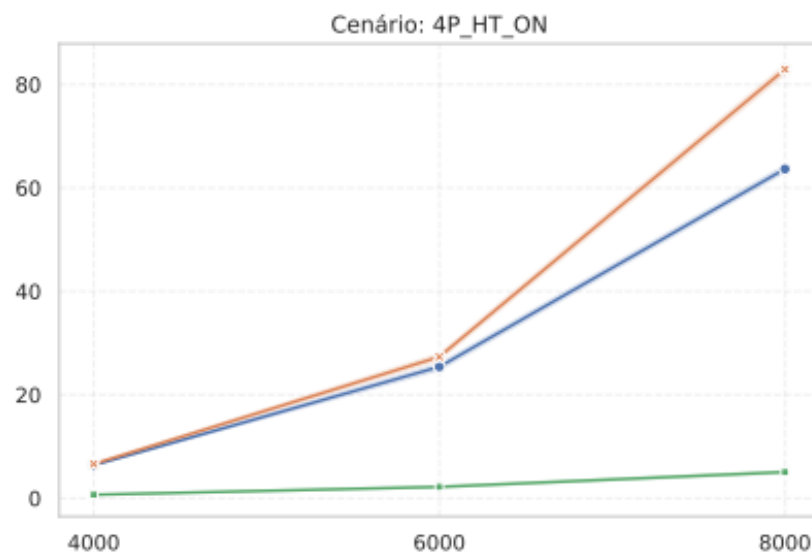
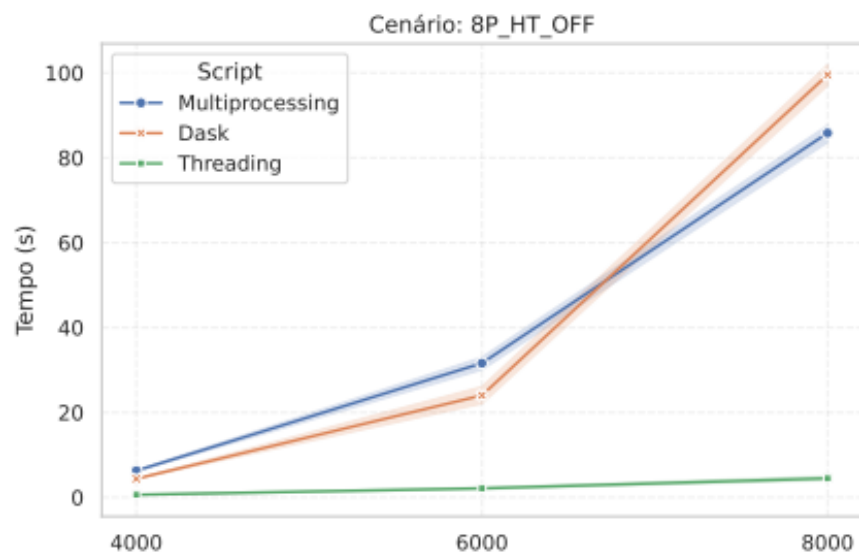
Threading:
Desempenho superior.

MultiP e Dask:
Semelhante.



MultiP
ligeiramente mais
eficiente (I.C.
95% sem
sobreposição)

Resultados: Por Cenários – Matriz 4000, 6000 e 8000

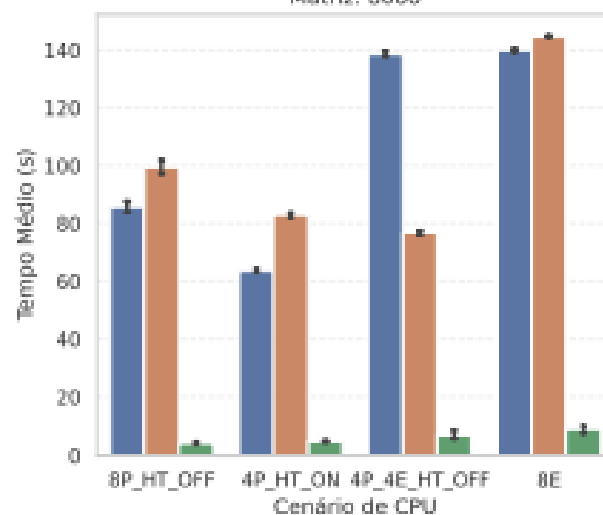
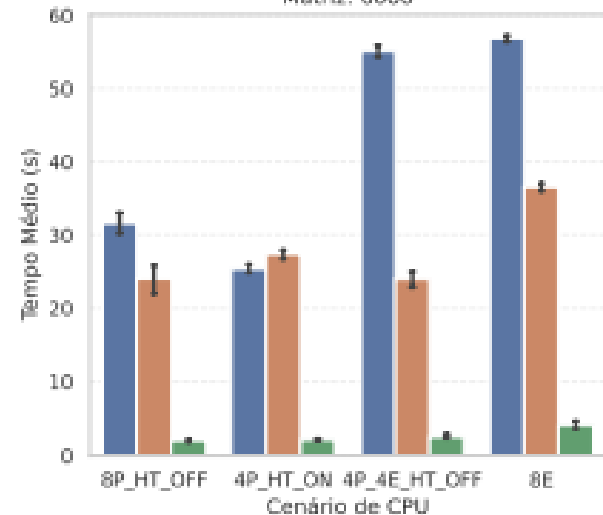
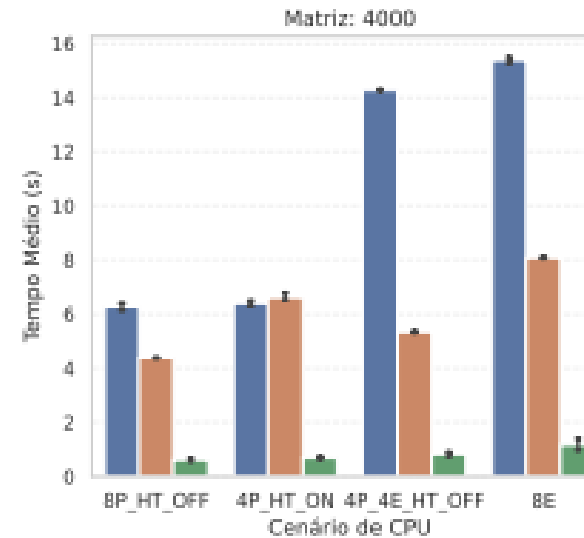
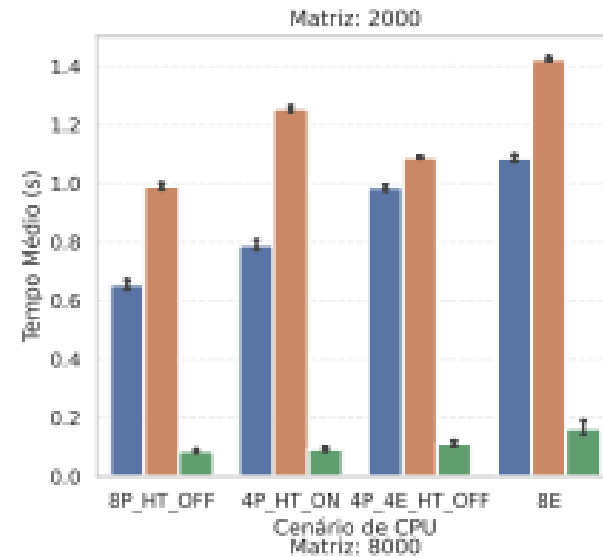
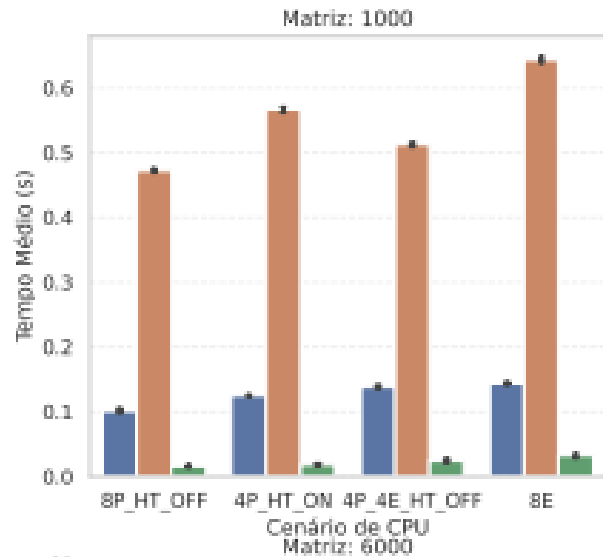


MultiP e Dask:
Inversão. Dask
melhor para 4000
e 6000

MultiP: Força
utilização de
núcleos.

Dask: Gerencia
Pool de núcleos.
Priorização de P-
Cores.

Resultados: Por Tamanhos matriz



Script

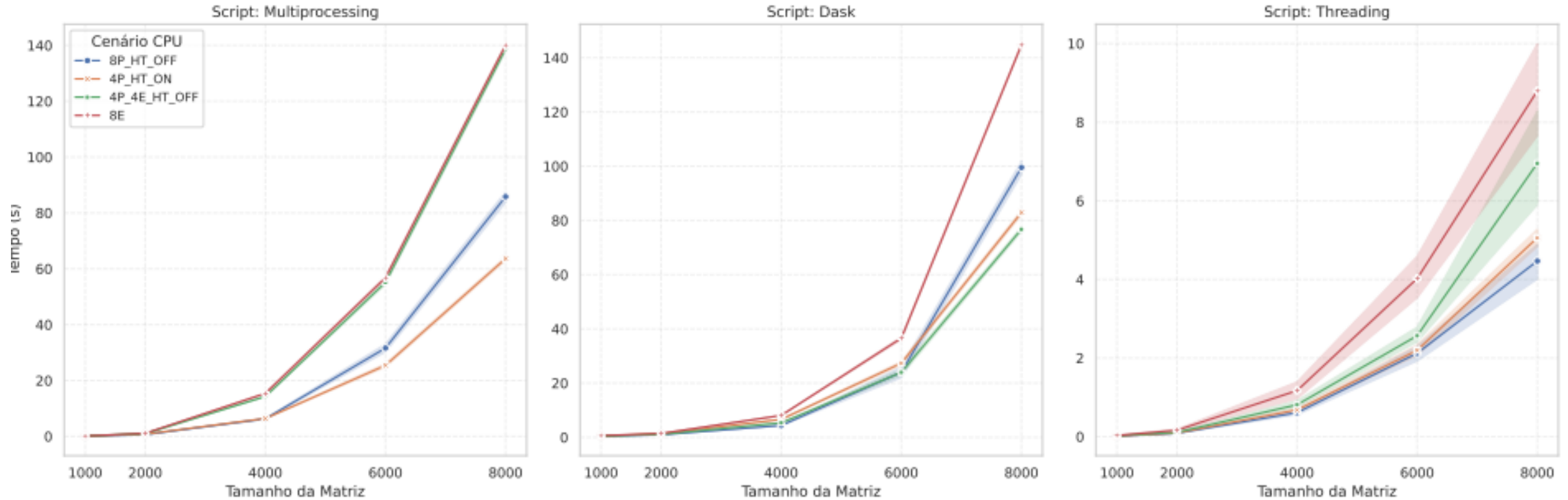
- Multiprocessing
- Dask
- Threading

Matriz 1000 e 2000: MultiP é melhor.

Matriz 4000 e 6000: Dask aumenta performance.

Matriz 8000: MultiP levemente melhor.

Resultados: Por Script



P-Cores: Menores tempos; **E-Cores:** Maiores tempos

Threading: Cenários com P-Cores estatisticamente iguais / Cenários com E-Cores estatisticamente iguais (I.C. 95%)

Reprodutibilidade

- Notebooks disponíveis no GitHub
- **Pilha de software:**
 - `pyenv shell 3.12.12`
 - `pip freeze > requirements_py3.12.txt`
 - `pyenv shell 3.14.0t`
 - `pip freeze > requirements_py3.14t.txt`
 - `apt-mark showmanual > system_packages.txt`
- **Reconstituição:**
 - `pip install -r requirements_py3.12.txt`
 - `pip install -r requirements_py3.14t.txt`
 - `sudo xargs apt-get install -y < system_packages.tx`

Conclusões

1. Utilizar P-Cores e/ou E-Cores tem impacto direto no desempenho
2. Python 3.14.0t, GIL free, eliminou gargalos críticos da serialização
 - Abordagem promissora para **computação paralela em Python** no futuro próximo.
3. Em arquiteturas híbridas a pinagem de núcleos pode não ser a melhor abordagem
 - Depende da aplicação, mas
 - O Scheduler do sistema é capaz de fazer boas escolhas de alocação

Referências

- [1] Dell. 2025. Desktop OptiPlex Micro. <https://www.dell.com/pt-br/shop/cty/pdp/spd/optiplex-7020-micro>.
- [2] João Luiz Grave Gross. 2025. Trabalho Final CMP223. <https://github.com/jlggross/TF-cmp223>.
- [3] Intel. 2021. Arquitetura híbrida de alto desempenho.
<https://www.intel.com/content/www/us/en/developer/articles/technical/hybrid-architecture.html>.
- [4] Intel. 2025. How Intel Technologies Boost Your CPU's Performance.
<https://www.intel.com/content/www/us/en/gaming/resources/how-intel-technologies-boost-cpu-performance.html>.
- [5] Ubuntu. 2025. ubuntu 24.03 LTS. <https://ubuntu.com/download/desktop>.

Obrigado!